

PART 1 · 为什么

03 问题背景

AI编程的质量困境与SDD运动的局限

04 核心论点

STDD的核心循环不依赖代码

PART 2 · 是什么

05 理论基础

质量模型·过程改进·知识管理·控制论

06-07 方法论

六阶段流程+经验复利机制

08 技术实现

Agent验证管线架构

PART 3 · 怎么用

09-10 金融场景

量化策略 · 风控审计 · 基金上线 · 数据报表

11-12 日常工作

官网维护 · 文档发布 · 客户管理 · 新人入职

13 技术运维

部署验证 · 配置管理 · CI/CD · 事故复盘

PART 4 · 价值与行动

14-17 效果评估

定量数据 · 成本效益 · 组织影响 · 框架对比

18-22 展望与行动

边界·未来·快速开始·结论

THE QUALITY CRISIS

AI 能写代码 但不能保证写对

38.2%

AI任务一次通过率
DAPLab 2026, n=2,847

14.7%

复杂多文件通过率
同上

65%

企业AI失败
因上下文漂移
Anthropic 2026

60-90%

编译通过但
语义错误
AWS Kiro 引述

行业共识：在AI生成代码前，先就"做什么"达成结构化一致 → 规范驱动开发(SDD)运动

THE BIG IDEA

三要素循环

不依赖代码



编程场景
GIVEN/WHEN/THEN
→ pytest 测试
→ 代码失败模式



非编程场景
前置条件/操作/预期
→ Agent CP 断言
→ 流程失败模式

质量不依赖执行者身份，依赖"预期是否被明确定义"和"结果是否被系统化验证"



软件质量模型

McCall三要素 · ISO 25010八特性 · Humphrey质量门。STDD的12类失败模式覆盖ISO 25010的6个质量特性。



过程改进理论

Deming PDCA循环 · CMMI五级成熟度 · Six Sigma DMAIC。STDD经验系统=自动化DMAIC循环。



知识管理模型

Nonaka SECI模型·Kolb学习循环。STDD经验FSM=可操作的SECI: 外化→组合→内化→社会化。



控制论基础

Wiener反馈控制·Kolmogorov复杂性。Spec=目标值，Test=测量值，Learn=纠偏。控制有效性不依赖被控对象。



人因工程

Parasuraman自动化层级 · Bainbridge悖论 · Endsley情境意识。STDD
Gate=自适应自动化。



信息与决策论

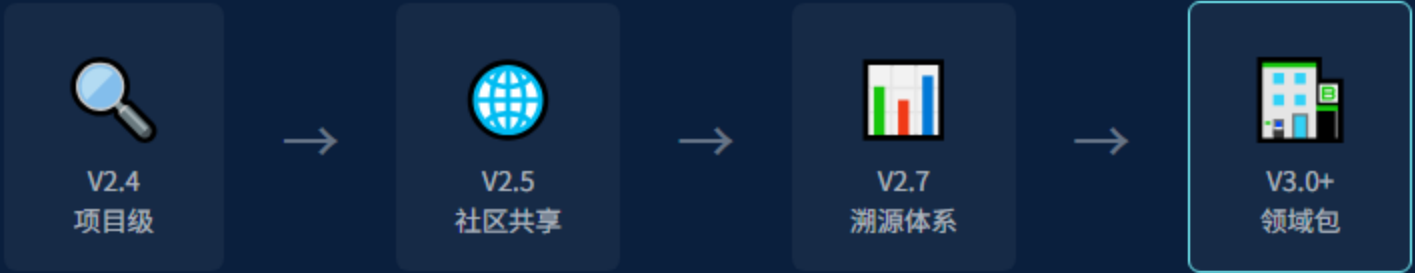
Spec精确度与AI输出不确定性的反比关系。锚定法L1-L4=逐步增加描述精确度降低不确定性。



六阶段的本质不依赖"代码"——每个阶段的抽象含义是通用的

经验复利

$E(n) = E(o) + \alpha n(1 - e^{(-\beta n)})$



金融包

策略 · 风控 · 基金 · 合规



医疗包

FDA/CE认证 · 数据完整性



法律包

GDPR · SOC2 · 等保合规

agent_spec.yaml — 机器可执行+人类可读

```
meta:
  task_id: deploy-staging
  system: production-server
steps:
  - id: CP-1
    description: 健康检查
    action: curl https://host/health
    assertions:
      - type: http_status
        expected: 200
      - type: stdout_contains
        expected: "ok"
  - id: CP-2
    action: docker compose up -d
    assertions:
      - type: exit_code
        expected: 0
rollback:
  steps:
    - docker compose down
    - docker compose up -d --no-build
```



6种断言类型

exit_code / stdout / stderr / http_status / file_exists / file_contains



双用途设计

机器可执行指令 = 人类可读SOP文档。永不漂移。



效率提升

部署验证: 15-20分钟 → 90秒 (-92%)



回滚机制

失败后不自动回滚，生成报告提示人工确认



风控规则审计

完整追溯链: 需求→proposal→Gate→test-report→archive。满足巴塞尔III和中国银保监会合规要求。传统邮件审批→结构化Gate确认。



基金产品上线

20+ CP标准化检查清单。跨托管/备案/销售/清算系统协调。遗漏率: 15%→0%。跨部门沟通: -60%。



数据报表验证

5 CP: 数据源完整性→交叉验证→异常检测→格式合规→发送。人工核对工作量-80%。



金融ROI分析

单次实盘部署潜在损失=X。STDD降低遗漏率90%。管理千万级资金的团队，ROI可达100:1以上。

LIVE DEMO: STDD 官网 V2.5

官网维护 + 文档发布

最成熟的非编程实践

官网 6 CP

链接有效性 · 中英双语 · 响应式 · 版本号 · 性能 · SEO。回滚率15%→0%。中英不同步2-3处→0。

文档 4 CP

版本号一致性 · 内部链接 · 代码可执行 · 中英段落。版本不一致-85%。新增CLI文档遗漏-90%。

-100%

发布回滚率

-85%

文档不一致

-100%

链接失效遗漏



客户需求管理

真实案例: 金融软件公司。客户需求→proposal→Gate锁定→验收。需求变更-77%。验收一次通过率33%→100%。返工31%→8%。满意度3.2→4.6/5。



新人Onboarding

15 CP标准化流程。入职5天→2天(-60%)。拟合曲线 $T(n) \approx 5.2 \times n^{(-0.38)}$ 。第10人 ≈ 2.1 天。年ROI 15:1。



会议决策追踪

技术决策不再依赖聊天记录。每个决策走proposal→design→archive。6个月后准确回忆: 口头30%→文档100%。



经验跨域迁移

官网中英不一致经验→文档中英检查CP。金融参数交叉验证→配置diff对比CP。场景越多，交叉越丰富。

PRODUCTION OPERATIONS

部署验证

5 CP · 90秒 · 13.3%拦截率

CP-1 镜像拉取 · CP-2 容器启动 · CP-3 健康检查 · CP-4 日志异常 · CP-5 回滚就绪

效率: 15min→90s (-92%)

30次部署拦截4次问题 (13.3%)

配置变更管理

5 CP: 快照→校验→生效→回归→回滚。20次变更0生产告警(此前10%)

CI/CD 质量门

7项自动检查。非代码项目自动切换检查维度。push触发全自动。

事故复盘

完整STDD流程。V2.7事故→2条经验→驱动V2.8的5项流程修复

策略遗漏率
15%→0%

文档不一致
每版本2-3处→0

新人上手
5天→2天

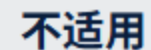
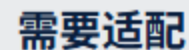
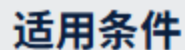
经验留存率
20%→100%

需求返工率
31%→8%

部署验证时间
15min→90s

9 大框架非编程适用性对比

STDD是唯一具备完整非编程支持 + 经验自学习 + 社区共享的框架



短期 (6个月)

领域经验包

金融/医疗/法律垂直领域预置agent_spec模板和经验库，社区贡献+官方审核。

🤖 AI辅助CP生成

自然语言描述→agent_spec.yaml自动生成。降低非技术用户的使用门槛。

多项目分析

基于project-index.yaml跨项目经验关联，识别领域级通用失败模式。

中长期 (6-24月)

🔗 多系统Agent验证

跨系统业务流程: 下单→支付→物流→通知。TEAM版核心能力。

🎯 自适应检查点

AI根据历史经验自动调整CP优先级。高频失败点→强制检查。长期PASS→降级快速检查。

经验效果量化

每条经验的经济价值模型。阻止问题的潜在损失=经验的ROI。量化经验库价值。

立即开始

1. 安装 STDD

```
git clone https://github.com/leonai42/stddev.git
cd stddev && python bin/stddev init
python bin/stddev install claude-code
```

2. 创建第一个非编程Change

```
mkdir -p changes/website-update
# 编写 proposal.md + agent_spec.yaml
python bin/stddev agent verify website-deploy
```

推荐入手路径

官网维护(6 CP, 最简单) → 文档发布(4 CP) → 新人入职(15 CP, 经验复利最明显)

STDD 的本质

不是一个编程工具
而是一套 让 AI 的输出
从 "不确定" 变为 "可预期"
的方法论

适用于任何 "AI 能执行、但需要质量保障" 的场景

GITHUB.COM/LEONAI42/STDD · MIT LICENSE · 小以AI实验室